

Assignment 2. PKI/TLS

Project Release: April 7, 2026 (Tuesday) KST

Deadline: April 17, 2026 (Friday) by 8:59:59pm KST

The code and other answers you submit must be entirely your own work. You may discuss this assignment in small groups with classmates, but you must write your code and solutions yourself. Please acknowledge any collaborators you discussed the assignment with and cite any references you used. You may consult published references, provided that you cite them appropriately (e.g., in program comments).

Solutions must be submitted via Blackboard.

Introduction

In this assignment, you will implement a simplified TLS interception system using OpenSSL. The primary goal of this assignment is to understand:

- how TLS is implemented,
- how TLS interception (man-in-the-middle proxying) is performed,
- and why such interception can introduce security risks.

TLS Interception (10pts)

Background

Transport Layer Security (TLS) is designed to provide end-to-end security between a client and a server. It ensures confidentiality, integrity, and authentication.

In the ideal model:

Client $\longleftrightarrow$ Server

the client establishes a secure channel directly with the server. The server presents a certificate, which is verified using a trusted Certificate Authority (CA). If the certificate is valid, the client assumes it is communicating with the legitimate server. This model assumes that there is no intermediary that can decrypt or modify the communication.

In practice, ‘TLS interception’ breaks this end-to-end model. Instead of one secure channel, there are two:

$$\text{Client} \longleftrightarrow \text{Proxy} \quad \text{and} \quad \text{Proxy} \longleftrightarrow \text{Server}$$

The proxy terminates the TLS connection from the client, decrypts the traffic, inspects or modifies it, and re-encrypts and send it to the server.

This effectively enables the proxy to access and modify data that was originally intended to be protected by TLS. Although such behavior violates the end-to-end security guarantees of TLS, interception is nonetheless widely deployed in practice for security monitoring and policy enforcement. In particular, enterprise firewalls analyze traffic for malware, antivirus systems inspect TLS communications, and content filtering mechanisms enforce organizational restrictions. The underlying rationale is that the increasing prevalence of encryption significantly limits the visibility of traditional inspection systems, preventing them from identifying potentially malicious content. Consequently, TLS interception is introduced as a mechanism to restore visibility into encrypted traffic, allowing these systems to continue functioning in an environment where most communications are encrypted.

Although TLS interception is intended to enhance security, it can in fact weaken the security guarantees originally provided by TLS. As shown by Durumeric et al. [1], many proxy systems introduce weaker cryptographic configurations than those that would have been negotiated in a direct end-to-end connection. For example, even if a client attempts to establish a connection using TLS 1.2 (the most recent version at the time of the study), an intercepting proxy may downgrade the connection on one side to TLS 1.0 when communicating with the server. More generally, the intermediary proxy may rely on outdated libraries, inconsistent implementations, or misconfigured policies, resulting in the use of weaker protocol versions, deprecated cryptographic algorithms, or even ciphers with known vulnerabilities. Consequently, rather than preserving or strengthening security, TLS interception may reduce the overall security of the connection and expose users to risks that would not have arisen in the original end-to-end setting.

If you are interested in this topic, I encourage you to read the paper [1] for further details; however, it is not directly related to the programming requirements of this assignment or to the exam.

Simple TLS Interception

Setup. You must develop and test your implementation inside the provided virtual environment, which is pre-configured to run the server binary. The username and password are both cse467. Download the appropriate VM image for your platform and run it. For Windows, Linux, and Intel-based Macs, download either the `.vdi` or `.ova` file. If you choose the `.vdi` file, create a new virtual machine in VirtualBox and select “Use an existing virtual hard disk file.” (the exact wording may vary depending on the VirtualBox version). You must also enable UEFI by navigating to Settings – System – Motherboard – enable EFI. If you choose the `.ova` file, import it into VirtualBox. For Apple Silicon (M1/M2) or Intel Mac users using UTM, download the `.zip` file, extract it, and open the included UTM image using UTM. Once the VM is running, you may access it via SSH using `cse467@localhost:8022` (e.g., `ssh -p 8022 cse467@localhost`). If SSH does not work, verify that the VM has a properly configured network interface (e.g., confirm that the interface is up and

that the VM has internet connectivity). You can find all the required files in the `pa` directory.

Guidelines. As illustrated above, the system consists of three entities: a client, a proxy, and a server. The server implementation is provided (binary), and your task is to implement the client and proxy according to the specification below. At a high level, the client communicates with the server by sending the content of a *text* file (see `input.txt`), and the server responds with the message “OK” along with a digital signature over that message. The communication follows a simple custom protocol over TLS, which is also reflected in the provided server implementation.

The protocol is defined as follows:

- **Client to Server (or Proxy):**
 - 4-byte unsigned integer (network byte order) indicating the length of the file
 - raw file content (exactly the specified number of bytes)
- **Server to Client:**
 - 4-byte unsigned integer: length of message
 - message bytes (the string “OK”)
 - 4-byte unsigned integer: length of signature
 - signature bytes (signature over the message)

You must follow this protocol exactly. In particular, you must correctly handle byte ordering (use `htonl` / `ntohl`) and ensure that all bytes are transmitted and received, even in the presence of partial reads and writes.

The proxy acts as a simple interception proxy. It monitors the application-layer data and performs a specific modification: if the substring “stopword” appears in the transmitted content, it removes all occurrences of this substring before forwarding the modified content to the server. The proxy must not modify any other part of the protocol (e.g., lengths must match the modified content).

In practice, TLS interception proxies are deployed transparently, and applications are typically unaware of their presence. For the purpose of this assignment, we emulate this behavior using different port numbers. The proxy listens on port 8443, while the server listens on port 9443. To simulate a direct connection (no proxy), the client connects to port 9443. To simulate a proxied connection, the client connects to port 8443. You can find example execution commands in the provided `Makefile`. Note that the provided `Makefile` produces compilation errors unless the source code is properly completed.

The server and the proxy each have their own certificate, but both certificates share the same subject name. However, they are signed by different root CA certificates. The client should be configured to trust both root CAs. All certificates are generated using the provided script `gen_certs.sh`. As a result, TLS handshakes will succeed in both the direct and proxied cases, even though the peer identity (in terms of public key) differs.

Your goal is to complete `client.c` and `proxy.c` such that they satisfy the following requirements.

Client Requirements. The client must:

- Accept four inputs: hostname (i.e., localhost), port number, trusted root CA certificates (i.e., `certs/ca_bundle.pem`), and input file name
- Establish a TLS connection to the given host and port (i.e., either proxy or server)
- Support both TLS 1.2 and TLS 1.3
- Load and trust both provided root CA certificates, so that the server (or proxy) certificate is accepted during the TLS handshake.
- Read the input file and send its contents using the defined protocol
- Receive the message and signature from the peer
- Extract the TLS peer certificate
- Extract the public key
- Verify the signature over the received message using the extracted public key
- Output (the template messages are already provided in `client.c`; you only need to ensure they are triggered correctly):
 - Used TLS version, used cipher, received message, and signature verification result.
 - “No Proxy exists.” if verification succeeds
 - “Proxy exists.” if verification fails

Note that verification success implies that the TLS peer is likely the original server, while failure suggests that the TLS peer (e.g., proxy) differs from the entity that generated the signature.

Proxy Requirements. The proxy must:

- Accepts six inputs: the proxy certificate (i.e., `certs/proxy.crt`), the proxy key (i.e., `certs/proxy.key`), the CA certificate (i.e., `certs/ca_server.pem`), the listening port (i.e., 8443), the server hostname (i.e., 127.0.0.1), and the server port (i.e., 9443).
- Listen on port 8443 and accept incoming TLS connections
- Support **TLS 1.2 only** for both incoming (client) and outgoing (server) connections
- Upon accepting a client connection, establish a separate TLS connection to the server (port 9443)
- Receive the file content from the client using the defined protocol
- Remove all occurrences of the substring “stopword”
- Recompute the length field accordingly
- Forward the modified content to the server
- Receive the server’s response (message + signature)

- Relay the response **without any modification** back to the client

The proxy must not perform any signature operations or certificate verification on application data. Its role is limited to interception and forwarding.

Implementation Notes. Skeleton codes for `client.c` and `proxy.c` are provided in the directory `pa2`. Some functions are already implemented. You are encouraged to use it, and most of your implementation can be completed by modifying the `main` function and the helper function `verify_signature_with_peer_cert`. However, you are not required to use the skeleton code, as long as your implementation satisfies all requirements and uses the OpenSSL library.

The provided server is intended only for functional testing. Additional test cases will be used during grading to verify correctness.

Hints. You can find useful APIs in the OpenSSL website (https://docs.openssl.org/3.0/man3/SSL_CTX_new/).

- Search `SSL_get_peer_certificate()`
- Search `X509_get_pubkey()`
- Search `EVP_DigestVerifyInit` / `Update` / `Final`
- Always check return values and error messages of OpenSSL APIs
- Implement helper functions to read/write exactly N bytes over TLS
- Be careful with memory management (free X509 and EVP structures)

Figure 1 shows example outputs of each component during execution.

Common Pitfalls.

- Forgetting to verify certificates during TLS handshake
- Incorrect handling of byte order (length fields)
- Assuming `SSL_read()` returns all requested bytes in one call
- Not updating the length after removing “stopword”
- Using the wrong public key for signature verification
- Memory leaks from OpenSSL objects

What to submit. Two source files: `client.c` and `proxy.c` and that satisfy the above client and proxy requirements. Your code must include clear, well-structured, and sufficiently detailed comments to explain the implementation.

Submission checklist

- `client.c`

```

└─ make run-client-proxy
./client 127.0.0.1 8443 certs/ca_bundle.pem input.txt
[client] connected to 127.0.0.1:8443
[client] TLS version: TLSv1.2
[client] cipher      : ECDHE-RSA-AES256-GCM-SHA384
[client] received message: OK
[client] signature verification failed.
[client] Proxy exists.

```

(a) Client output through the proxy

```

└─ make run-client-direct
./client 127.0.0.1 9443 certs/ca_bundle.pem input.txt
[client] connected to 127.0.0.1:9443
[client] TLS version: TLSv1.3
[client] cipher      : TLS_AES_256_GCM_SHA384
[client] received message: OK
[client] signature verification succeeded.
[client] No proxy exists.

```

(b) Client output with direct server connection

```

└─ make run-proxy
./proxy certs/proxy.crt certs/proxy.key certs/ca_server.pem 8443 127.0.0.1 9443
[proxy] listening on 8443
[proxy] client->proxy TLS version: TLSv1.2
[proxy] client->proxy cipher      : ECDHE-RSA-AES256-GCM-SHA384

```

(c) Proxy output

```

└─ make run-server
./server certs/server.crt certs/server.key 9443
[server] listening on 9443
[server] TLS version: TLSv1.3
[server] cipher      : TLS_AES_256_GCM_SHA384
[server] received content:
hello stopword world

[server] TLS version: TLSv1.2
[server] cipher      : ECDHE-RSA-AES256-GCM-SHA384
[server] received content:
hello world

```

(d) Server output

Figure 1: Example outputs of the client, proxy, and server.

- proxy.c
- **(Optional)** Makefile and README: submit these only if you modified the provided **Makefile** or changed the execution method. In such cases, you must provide a clear and reasonable justification for the changes.

References

- [1] Zakir Durumeric, Zane Ma, Drew Springall, Richard Barnes, Nick Sullivan, Elie Bursztein, Michael D Bailey, J Alex Halderman, and Vern Paxson. The security impact of https interception. In *NDSS*, 2017.